Author-Written Test Sets Overstate Lightweight Prompt-Injection Detectors: A Case Study with an Offline Guardrail Toolkit

Ashish Dhillon
[Affiliation]
dhillon.ashish15@gmail.com

Draft, 1 November 2026

Abstract

Lightweight, dependency-free guardrails for LLM applications are attractive because they run offline and add about a millisecond per request. We ask how well their headline numbers transfer. We built *guardkit*, an open-source toolkit with rule-based and logistic-regression prompt-injection detectors and checksum-validated PII detection. On two hand-written evaluation sets the detector reaches precision 0.94 and recall 0.67 on unseen phrasings (recall 0.85 on a development set). On a public in-the-wild jailbreak dataset (15,140 prompts, 1,405 jailbreaks) the same detector family is much weaker: trained on the earlier 65% of the data and tested on the later part, a bag-of-words classifier reaches AUC 0.74, and regular expressions reach recall 0.05. Using an open benchmark harness (*injection-bench*) with bootstrap confidence intervals, a word-plus-character n-gram TF-IDF classifier reaches AUC 0.78 but recalls only 9% of jailbreaks at a 1% false-positive rate, while prompt length alone gives AUC 0.59. A pretrained DeBERTa-v3 prompt-injection classifier scores no better on the same test (AUC 0.75, recall 9% at 1% false positives), with an interval that overlaps those of the lightweight models. We also find that 5.2% of the dataset's benign prompts contain the literal phrase "ignore all previous instructions", so that phrase is an unreliable signal in user prompts although it is a strong signal in third-party documents. Separately, checksum validation (Luhn for cards, Verhoeff for India's Aadhaar numbers) removes about 90% of false positives on random digit strings and reduced 166 card-shaped strings in real prompts to 9. We recommend external, time-split evaluation with recall reported at fixed false-positive rates.

1 Introduction

Guardrails for LLM applications sit before and after the model to block prompt injection, redact personal data and keep secrets out of responses. Many deployments want a first layer that is fast, deterministic and runs without a model download or API call. Rule sets and small linear classifiers meet that need, but they are usually validated on examples written by their authors. Such examples share vocabulary with the rules and the training data, so they can overstate behaviour on real traffic.

This report documents one such tool and measures the gap directly. Our contributions are modest and empirical: (1) an open-source, zero-ML-dependency toolkit whose trained classifier ships as a JSON file of about 60 KB; (2) a measurement of the gap between author-written and external,

time-split evaluation for the same detector family; (3) an open benchmark harness that reports recall at fixed false-positive rates with confidence intervals and shortcut baselines; (4) an observation about label semantics, where a phrase that signals an attack inside a retrieved document is common in benign user-prompt templates; and (5) a small measurement of how much checksum validation reduces false positives in PII detection.

2 The toolkit

guardkit applies configurable guards to input and output text. Each guard takes one of three actions: block, redact or warn. Policies are written in YAML. The PII guard covers email, phone numbers (India and international), payment cards validated with the Luhn algorithm, Aadhaar numbers validated with the Verhoeff algorithm, PAN, IPv4, US SSN and PIN codes; cue-based name and address heuristics are opt-in. Other guards detect secrets (well-known credential formats and high-entropy assignments), banned topics and malformed JSON. The injection guard has (a) ten regular-expression rules applied after Unicode normalisation (NFKC, zero-width removal, case folding) and (b) a logistic-regression classifier over hashed word unigrams and bigrams (65,536 buckets) implemented in pure Python. Two classifiers ship: `injection`, trained on 2,800 synthetic template examples, and `jailbreak`, trained on public data (Section 3).

3 Evaluation setup

Author-written sets. We wrote two labelled sets of short texts. The development set (149 examples, 75 attacks) was used while building the system and is therefore optimistic. The held-out set (102 examples, 51 attacks and 51 benign) was written after training-data generation and evaluated once before a single data-bug fix: a padding phrase that appeared only in attack training examples made "thanks!" look malicious. After that fix both sets were re-scored. Both sets and the training-data generator were written by the same author.

External data. We use the public in-the-wild jailbreak dataset of Shen et al. [1] (MIT licence), with 1,405 jailbreak and 13,735 regular prompts from Reddit, Discord and websites collected during 2022–2023. We first scored the detectors zero-shot on all 15,140 prompts. We then trained a second classifier with a temporal split: prompts created before 4 July 2023 form the training set (8,038 prompts, 758 jailbreaks after removing near-duplicates by a normalised 300-character prefix) and later prompts form the test set (5,497 prompts, 398 jailbreaks, 7.2% prevalence), with any test prompt whose prefix occurs in training removed. Training uses all jailbreaks and four randomly chosen benign prompts per jailbreak. Hyperparameters were fixed in advance (12 epochs of stochastic gradient descent, learning rate 0.5 with decay 0.85, L2 penalty 10^{-5}) and the test set was evaluated once. We report one operating point at the 0.5 threshold and one whose threshold is set on training negatives so that 1% of them are flagged.

Benchmark harness. We re-implemented the split, metrics and baselines in an open harness, injection-bench. It drops the 218 prompts without a timestamp, which yields 7,895 training and 5,503 test prompts (398 jailbreaks). It reports AUC and recall at fixed false-positive rates with 95% bootstrap intervals (200 resamples) and precision at a stated prevalence. For the fixed-rate metrics

the threshold is set on the *test* negatives, so they measure ranking quality at a false-positive budget, not a deployable operating point.

4 Results

4.1 Author-written sets

Set	Detector	Precision	Recall	F1
Held-out (n=102)	regex rules	1.00	0.02	0.04
	classifier (`injection`)	0.94	0.67	0.78
	rules + classifier	0.94	0.67	0.78
Development (n=149)	regex rules	1.00	0.24	0.39
	rules + classifier	1.00	0.85	0.92

Table 1: Author-written sets. The rules almost never fire falsely but miss paraphrases; the classifier recovers about two thirds of unseen ones.

4.2 External data, zero-shot

On all 15,140 prompts, the regex rules reach recall 0.134 at a false-positive rate of 0.074, the `injection` classifier 0.311 at 0.123, and the two combined 0.394 at 0.182. Detectors tuned on author-written data flag a large share of ordinary prompts and still miss most jailbreaks.

4.3 External data, time-split

Detector	Recall	FPR	Precision
regex rules	0.048	0.026	0.13
classifier, threshold 0.5	0.420	0.126	0.21
classifier, threshold at 1% training FPR	0.204	0.033	0.33

Table 2: Time-split test (5,497 prompts, 398 jailbreaks). Classifier AUC is 0.739.

At this prevalence a detector at 12.6% FPR produces about four false alarms per true detection. Scoring costs 0.5–1.2 ms per prompt on one CPU core.

4.4 Baselines and confidence intervals

Table 3 compares guardkit with simple baselines under the harness.

Prompt length alone gives AUC 0.59, so the data contain a mild but not dominant length shortcut. Classical bag-of-words models plateau at AUC 0.74–0.78 and recall roughly one jailbreak in ten at a 1% false-positive rate; adding character n-grams improves AUC but not low-FPR recall. The pretrained DeBERTa classifier lands in the same range (AUC 0.751, interval 0.72–0.78) and recalls 9% of jailbreaks at 1% FPR; its recall at 5% FPR (0.344, interval 0.28–0.39) is the highest in the table, but the interval overlaps that of the TF-IDF model (0.23–0.31), and it is roughly 60 to 100

Detector	AUC (95% CI)	Recall @1% FPR	Recall @5% FPR	Prec. @1% FPR
length only	0.587 (0.56–0.61)	0.020 (0.01–0.04)	0.080	0.10
keyword list	0.653 (0.63–0.68)	0.053 (0.03–0.07)	0.214	0.22
hashed uni/bigram LR	0.742 (0.72–0.77)	0.113 (0.08–0.16)	0.264	0.37
TF-IDF word+char LR	0.779 (0.76–0.80)	0.093 (0.07–0.14)	0.274	0.33
guardkit rules	0.511 (0.50–0.52)	0.000 (0.00–0.00)	0.048	0.00
guardkit classifier	0.739 (0.71–0.76)	0.118 (0.08–0.18)	0.261	0.38
DeBERTa-v3 (pretrained)[†]	0.751 (0.72–0.78)	0.090 (0.05–0.13)	0.344	0.32

Table 3: injection-bench results. Precision assumes 5% prevalence. Thresholds for the fixed-FPR columns are set on test negatives. [†]`protectai/deberta-v3-base-prompt-injection-v2`, scored zero-shot; it was not trained on our split and its training data may include similar public prompts.

times slower per item (49 ms on a Colab runtime whose hardware we did not record, versus 0.5–2.6 ms on CPU for the linear models). Overlapping unpaired intervals do not establish equivalence, and we did not store per-item scores for a paired test in this run; the harness now supports one. The guardkit classifier's AUC matches Section 4.3 (0.739); the benchmark's test set differs from that section's by six prompts because undated prompts are dropped.

4.5 A label-semantics finding

In the regular-prompt set, 721 of 13,735 prompts (5.2%) contain a literal "ignore all previous instructions" or a close variant. These are ordinary templates, for example an instruction to ignore previous instructions followed by a role description for a coding teacher. The regex override rule, which also matches looser variants, accounts for 1,000 of the 1,022 regular prompts the rules flag. The same phrase is a strong signal inside a retrieved document or tool output, because the user did not write it. Detectors therefore need to know where text came from, and one phrase list cannot serve both contexts.

4.6 PII checksum validation

Random 12-digit strings starting 2–9 pass the Verhoeff check 9.9% of the time and random 16-digit strings pass Luhn 10.0% of the time (200,000 samples each), so checksum validation removes about 90% of false positives from numeric look-alikes, while all 5,000 generated valid Aadhaar numbers were kept. On the 15,140 real prompts the regular expression for card-shaped strings (13–19 digits) matched 166 strings and 9 survived the Luhn check; for Aadhaar-shaped strings 4 matched and 1 survived. The data have no PII labels, so we cannot report precision.

5 Discussion

The author-written sets and the external data disagree for ordinary reasons. The authors' examples share vocabulary with the authors' rules and training generator. Real jailbreaks are long, varied role-play prompts, and real benign prompts include long system-style templates that look similar, leaving a hashed bag-of-words model little to separate them. We evaluated one pretrained transformer classifier zero-shot, and it did not separate from the linear models. Fine-tuning a transformer on the training split, other pretrained detectors and model-based judges remain untested, and we make

no claim about them. We recommend: evaluate on external, time-split data and report recall at fixed low false-positive rates; report precision at a stated prevalence, since a 12% false-positive rate is unusable at 7% prevalence; treat provenance as a feature, with different policies for user prompts and third-party content; and use checksum validation wherever an identifier has one.

6 Limitations

All hand-written sets and the synthetic training data were produced by one author, who also designed the rules. The external dataset labels jailbreak intent, not prompt injection in the document-embedded sense, and its benign class is not guaranteed free of jailbreaks. The temporal split covers one year and one data source. The learned models evaluated are linear plus one pretrained transformer scored zero-shot, so the results do not characterise fine-tuned transformers or LLM judges. The PII experiment has no ground-truth labels for real data. Latency was measured on one machine.

7 Reproducibility

Code, data scripts and results are public: guardkit (https://github.com/ashishdhillon/guardkit), injection-bench (https://github.com/ashishdhillon/injection-bench) and bharatid, the Indian-identifier validators (https://github.com/ashishdhillon/bharatid). `python scripts/evaluate.py` reproduces Table 1, and the scripts `eval_external.py`, `train_jailbreak.py` and `pii_checksum_experiment.py` reproduce Sections 4.2, 4.3 and 4.6. `injectbench run` reproduces Table 3. Seeds are fixed.

Acknowledgements and AI-use statement

Code, experiments and a first draft of this report were prepared with assistance from Claude (Anthropic). The author reviewed the results and is responsible for the content.

References

[1] X. Shen, Z. Chen, M. Backes, Y. Shen, Y. Zhang. "Do Anything Now": Characterizing and Evaluating In-The-Wild Jailbreak Prompts on Large Language Models. *ACM CCS*, 2024. arXiv:2308.03825. Data: https://github.com/verazuo/jailbreak_llms.

[2] J. Verhoeff. Error Detecting Decimal Codes. *Mathematical Centre Tracts* 29, 1969.

[3] H. P. Luhn. Computer for Verifying Numbers. US Patent 2,950,048, 1960.